

Basic Thread Synchronization

- Sección Crítica
- Synchronized
- Locks

Programación Concurrente y Paralela

2026

Dr. Ing. Ventre, Luis O.

Ing. Ludemann, Mauricio

Ing. Carranza, Agustín

Basic Thread Synchronization

- Una de las situaciones más comunes en programas concurrentes ocurre cuando más de un hilo en ejecución comparten un recurso.
 - En aplicaciones concurrentes es común que varios hilos escriban y lean los mismos datos/variables (ej. DB).
 - El acceso compartido a estos recursos pueden provocar errores o inconsistencia.

Se deben implementar mecanismos para evitar esto.

Basic Thread Synchronization

- La solución a estos problemas viene con el concepto de ***SECCION CRITICA (SC)***.
 - Una **SC** es un bloque de código o una parte del programa donde se accede a un recurso y NO puede ser ejecutada por más de un hilo al mismo tiempo.
 - Para facilitar la implementación de una **SC**, java ofrece mecanismos de sincronización.
 - Cuando un hilo quiere acceder a una SC utiliza uno de los mecanismos de sincronización provistos (para saber si hay otro hilo utilizando ese recurso).

Basic Thread Synchronization

- Cuando más de un hilo están esperando que otro finalice la ejecución dentro de la **SC**, la JVM selecciona uno y el resto espera su turno.
- Veremos el concepto de la palabra reservada

SYNCHRONIZED

Synchronizing a Method

- En el siguiente ejemplo se utiliza uno de los métodos básicos de sincronización.
- El uso de la palabra clave **synchronized** para controlar el acceso concurrente a:
 - Un método
 - Un bloque de código.
- Solo un hilo en ejecución puede acceder al mismo tiempo a un método/código que tenga en su definición la palabra **synchronized**.

Synchronizing a Method

- Cada método **synchronized** es una sección crítica.
- Java permite la ejecución de solo una método sincronizado (SC) por instancia de objeto. *
- Si otro hilo intenta acceder a cualquier otro método declarado **synchronized** del mismo objeto:
Será **suspendido** hasta que el primer hilo finalice su ejecución*

Synchronizing a Method

- Los **métodos estáticos** tienen otro comportamiento. Solo un hilo puede acceder un método estático **synchronized**; pero otro hilo puede acceder un método no estático **synchronized**. Si ambos métodos modifican mismas variables puede resultar inconsistencia.

Synchronizing a Method

Ejemplo

- Una playa de estacionamiento, que posee:
 - **Caja**, contabiliza el dinero de los vehículos
 - **Estadística**, posee la cantidad de autos y motos que hay en la playa de estacionamiento. Cuando un auto sale se encarga de ejecutar el pago en la Caja.
 - **Sensor**, detecta los vehículos que ingresan y salen de la playa, y actualiza el Estado.

Synchronizing a Method

Paso a Paso:

- Se crea la clase `ParkingCash` que posee dos atributos privados, el costo de un vehículo y el total de dinero recaudado.
- Implementar el método `vehiclePay()` que incrementará el valor del dinero recaudado
 - *Solo un auto a la vez puede acceder a la caja, por lo tanto este método deberá estar sincronizado.*
- Implementar el método `close()` para cerrar la caja e imprimir el total recaudado

Synchronizing a Method

Paso a Paso:

- Se crea la clase `ParkingStats` que posee:
 - Dos atributos, uno con la cantidad de autos y el otro con la cantidad de motos en la playa
 - Dos atributos del tipo `Object`, que van a servir para sincronización de bloques de Código. Uno para la entrada y salida de los autos y el otro para la entrada y salida de las motos
 - Un atributo más que es la referencia al `ParkingCash`
- Implementar los métodos *`carComeIn`*, *`carGoOut`*, *`motoComeIn`* y *`motoGoOut`*

Synchronizing a Method

Paso a Paso:

- Cada uno de los métodos debe poder incrementar/decrementar el valor de la cantidad de vehículos de manera segura
 - Para ello, se puede sincronizar un bloque de código, utilizando los objetos creados como llaves
- Implementar los métodos *getNumberCars* y *getNumberMotorcycles*
 - También deben leer las variables de cantidades de vehículos de manera segura sincronizandose

Synchronizing a Method

Paso a Paso:

- Se crea la clase Sensor que tiene un atributo que es la referencia al Estado de la playa
- Sensor implementa la interfaz Runnable, por lo tanto tiene que sobrescribir el método *run()*
- Cada hilo que ejecute Sensor, va a hacer cierta lógica donde entran y salen autos y motos de la playa.

Synchronizing a Method

Paso a Paso:

- Crear la clase Main donde:
 - Se crea la Caja de la playa
 - Se crea el Estado de la playa
 - Se crean n Sensores y n hilos para ejecutar los Sensores
 - Se ejecutan todos los hilos
 - El main ejecuta un *join()* a cada hilo
 - Se cierra la caja y se imprime por pantalla el total de autos y motos que quedaron en la playa (va a depender de la implementación del Sensor)

Synchronizing a Method

Como funciona:

- Se incrementa y decrementa la cantidad de autos y motos en igual cantidad de veces.
- Es esperable que el valor final de la cantidad de autos y motos sea 0.
- Hemos intentado forzar un error usando una variable temporal e introduciendo un delay.

Synchronizing a Method

Como funciona:

- Cada auto y moto que sale de la playa incrementa la variable de recaudación.
- Es esperable que al finalizar el programa, el total de recaudación sea el total de vehículos que salieron de la playa multiplicado por el costo
- **Si quiere observar los problemas de la concurrencia elimine las palabras synchronized de métodos y bloques de código.**

Synchronizing a Method

- El código:

```

Main.java × ParkingCash.java × ParkingStats.java × Sensor.java ×
1 package com.packtpub.java9.concurrency.cookbook.chapter02.recipe01.parking;
2
3 public class ParkingCash {
4
5     private static final int cost=2;
6
7     private long cash;
8
9     public ParkingCash() {
10         cash=0;
11     }
12
13     public synchronized void vehiclePay() {
14         cash+=cost;
15     }
16
17     public void close() {
18         System.out.printf("Closing accounting");
19         long totalAmmount;
20         synchronized (this) {
21             totalAmmount=cash;
22             cash=0;
23         }
24         System.out.printf("The total ammount is : %d",totalAmmount);
25     }
26 }
27
```


Synchronizing a Method

- El código:

```
Main.java x ParkingCash.java x ParkingStats.java x Sensor.java x
1 package com.packtpub.java9.concurrency.cookbook.chapter02.recipe01.parking;
2
3 public class ParkingStats {
4
5     /**
6      * This two variables store the number of cars and motorcycles in the parking
7      */
8     private long numberCars;
9     private long numberMotorcycles;
10
11     /**
12      * Two objects for the synchronization. ControlCars synchronizes the
13      * access to the numberCars attribute and controlMotorcycles synchronizes
14      * the access to the numberMotorcycles attribute.
15      */
16     private final Object controlCars, controlMotorcycles;
17
18
19     private ParkingCash cash;
20     /**
21      * Constructor of the class
22      */
23     public ParkingStats (ParkingCash cash) {
24         numberCars=0;
25         numberMotorcycles=0;
26         controlCars=new Object();
27         controlMotorcycles=new Object();
28         this.cash=cash;
29     }
30
31
32     public void carComeIn() {
33         synchronized (controlCars) {
34             numberCars++;
35         }
36     }
37
38     public void carGoOut() {
39         synchronized (controlCars) {
40             numberCars--;
41         }
42         cash.vehiclePay();
43     }
44 }
```

Synchronizing a Method

- El código:

```
Main.java × ParkingCash.java × ParkingStats.java × Sensor.java ×
1 package com.packtpub.java9.concurrency.cookbook.chapter02.recipe01.parking;
2
3 import java.util.concurrent.TimeUnit;
4
5 public class Sensor implements Runnable {
6
7     private ParkingStats stats;
8
9     public Sensor(ParkingStats stats) {
10         this.stats = stats;
11     }
12
13     @Override
14     public void run() {
15         for (int i = 0; i < 10; i++) {
16             stats.carComeIn();
17             stats.carComeIn();
18             try {
19                 TimeUnit.MILLISECONDS.sleep(50);
20             } catch (InterruptedException e) {
21                 e.printStackTrace();
22             }
23             stats.motoComeIn();
24             try {
25                 TimeUnit.MILLISECONDS.sleep(50);
26             } catch (InterruptedException e) {
27                 e.printStackTrace();
28             }
29             stats.motoGoOut();
30             stats.carGoOut();
31             stats.carGoOut();
32         }
33     }
34
35 }
```

Synchronizing a Method

- El código:

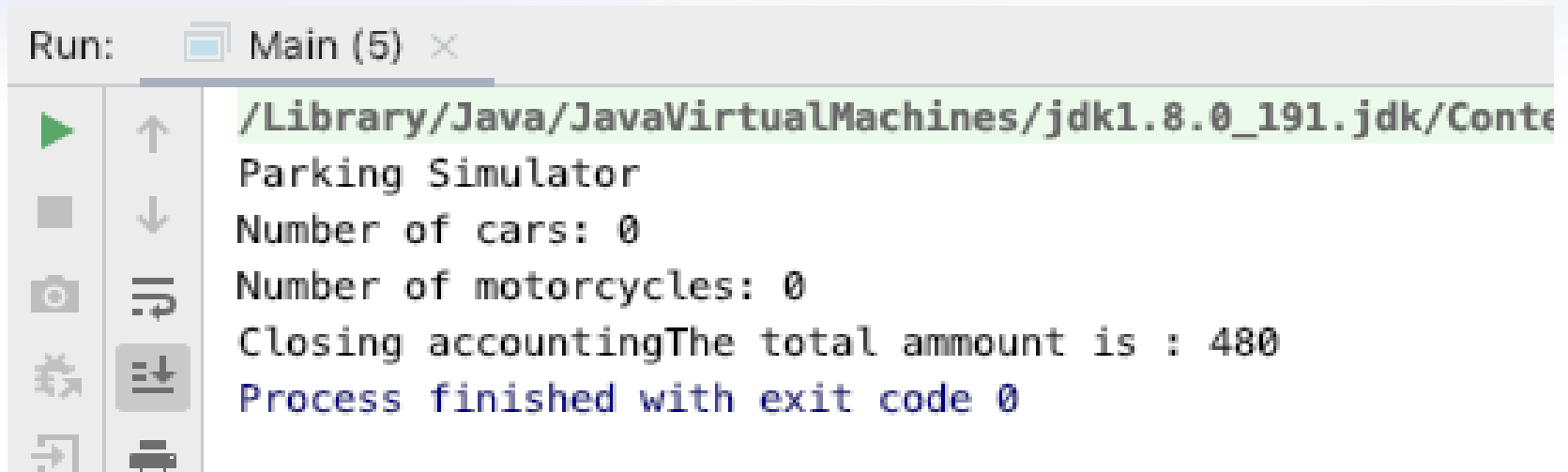
```
Main.java x ParkingCash.java x ParkingStats.java x Sensor.java x
1 package com.packtpub.java9.concurrency.cookbook.chapter02.recipe01.main;
2
3 import ...
4
5
6
7 public class Main {
8
9     public static void main(String[] args) {
10
11         ParkingCash cash = new ParkingCash();
12         ParkingStats stats = new ParkingStats(cash);
13
14         System.out.println("Parking Simulator");
15         int numberSensors=2 * Runtime.getRuntime().availableProcessors();
16         Thread[] threads =new Thread[numberSensors];
17         for (int i = 0; i < numberSensors; i++) {
18             Sensor sensor=new Sensor(stats);
19             Thread thread=new Thread(sensor);
20             thread.start();
21             threads[i]=thread;
22         }
23
24         for (int i=0; i< numberSensors; i++) {
25             try {
26                 threads[i].join();
27             } catch (InterruptedException e) {
28                 e.printStackTrace();
29             }
30         }
31
32         System.out.printf("Number of cars: %d\n", stats.getNumberCars());
33         System.out.printf("Number of motorcycles: %d\n", stats.getNumberMotorcycles());
34         cash.close();
35     }
36 }
```

Synchronizing a Method

- En la implementación del Sensor tenemos 10 veces la siguiente secuencia por cada Sensor:
 - Dos autos que entran
 - Una moto que entra
 - Una moto que sale
 - Dos autos que salen
- Total de autos esperados al finalizar: 0
- Total de motos esperadas al finalizar: 0
- Total de recaudación esperada con 8 Sensores:
 $8\text{sensores} * 10\text{veces} * 3\text{vehiculos} * 2\text{costo} = \mathbf{480}$

Synchronizing a Method

- La salida con **synchronized**:



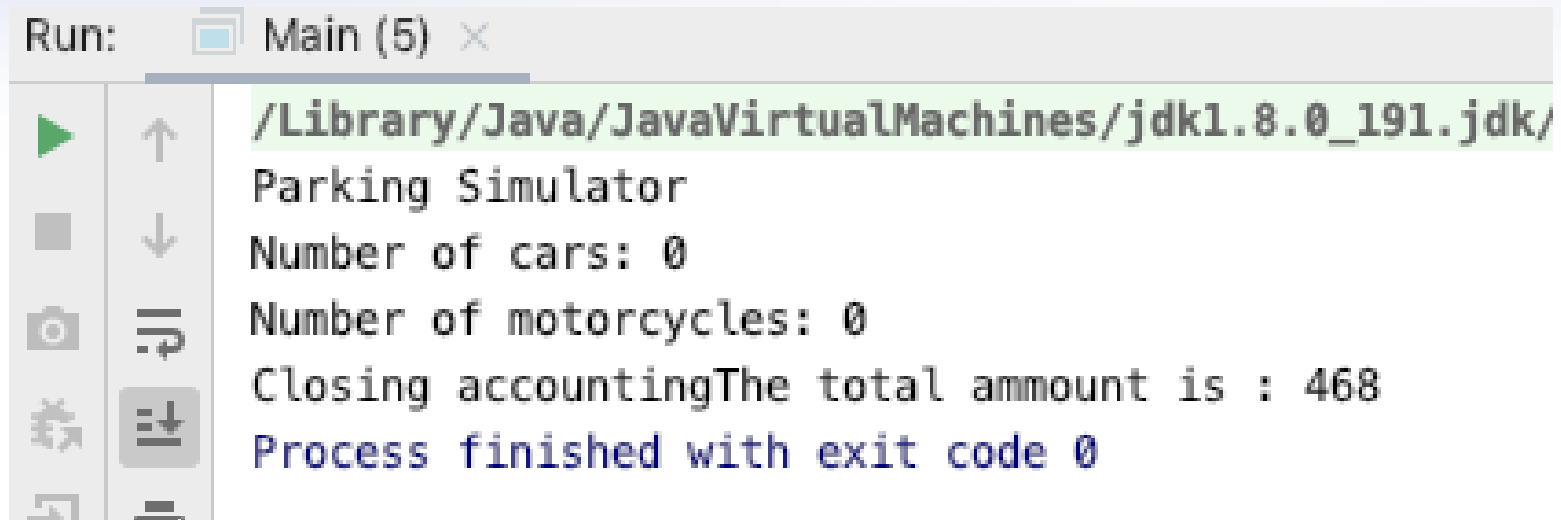
The screenshot shows a Java IDE's Run console window. The title bar reads "Run: Main (5) x". The console output is as follows:

```
/Library/Java/JavaVirtualMachines/jdk1.8.0_191.jdk/Contents/Resources/Java  
Parking Simulator  
Number of cars: 0  
Number of motorcycles: 0  
Closing accountingThe total ammount is : 480  
Process finished with exit code 0
```

The output lines are: `/Library/Java/JavaVirtualMachines/jdk1.8.0_191.jdk/Contents/Resources/Java`, `Parking Simulator`, `Number of cars: 0`, `Number of motorcycles: 0`, `Closing accountingThe total ammount is : 480`, and `Process finished with exit code 0`. The IDE interface includes a toolbar with icons for running, stepping through code, and other debugging actions.

Synchronizing a Method

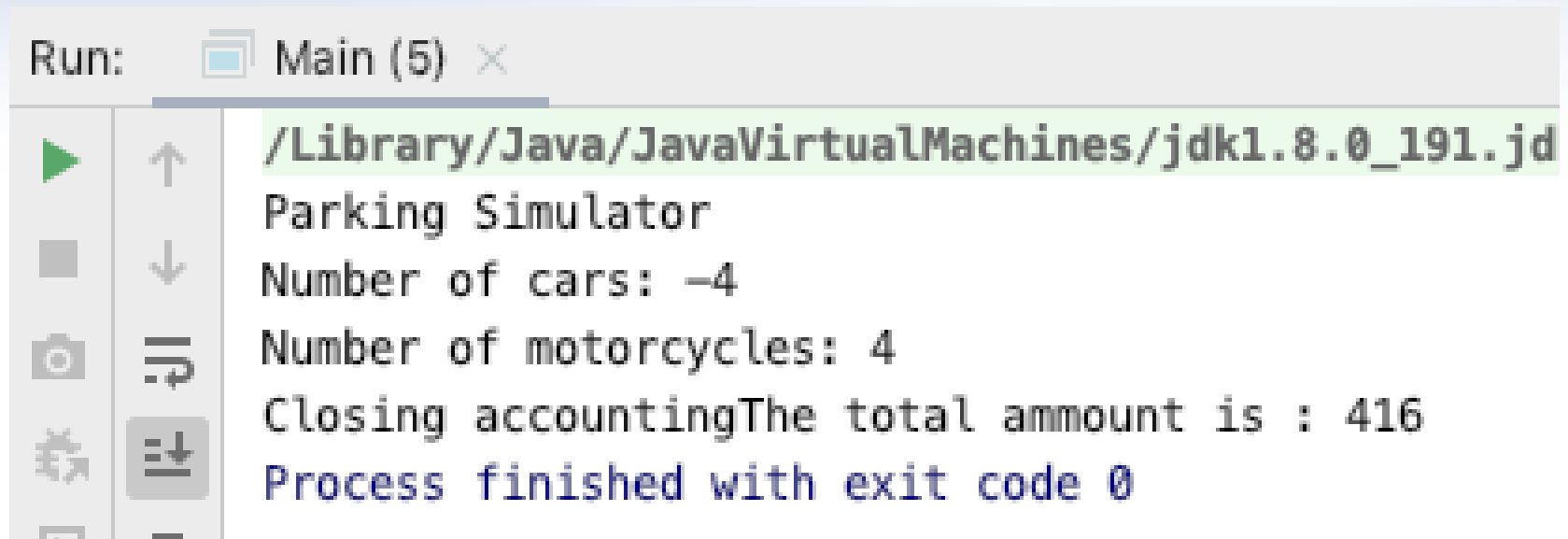
- La salida sin **synchronized** en *vehiclePay()*



```
Run: Main (5) ×  
/Library/Java/JavaVirtualMachines/jdk1.8.0_191.jdk/  
Parking Simulator  
Number of cars: 0  
Number of motorcycles: 0  
Closing accountingThe total ammount is : 468  
Process finished with exit code 0
```

Synchronizing a Method

- La salida sin ningún **synchronized**



```
Run: Main (5) ×  
/Library/Java/JavaVirtualMachines/jdk1.8.0_191.jd  
Parking Simulator  
Number of cars: -4  
Number of motorcycles: 4  
Closing accountingThe total ammount is : 416  
Process finished with exit code 0
```

Synchronizing a Method

- La palabra **synchronized** puede utilizarse para sincronizar un método o un bloque de Código
- La palabra clave synchronized penaliza la performance. Debe utilizarse solo en los casos necesarios.

SC as short as possible

Arranging independent attributes in synchronized classes

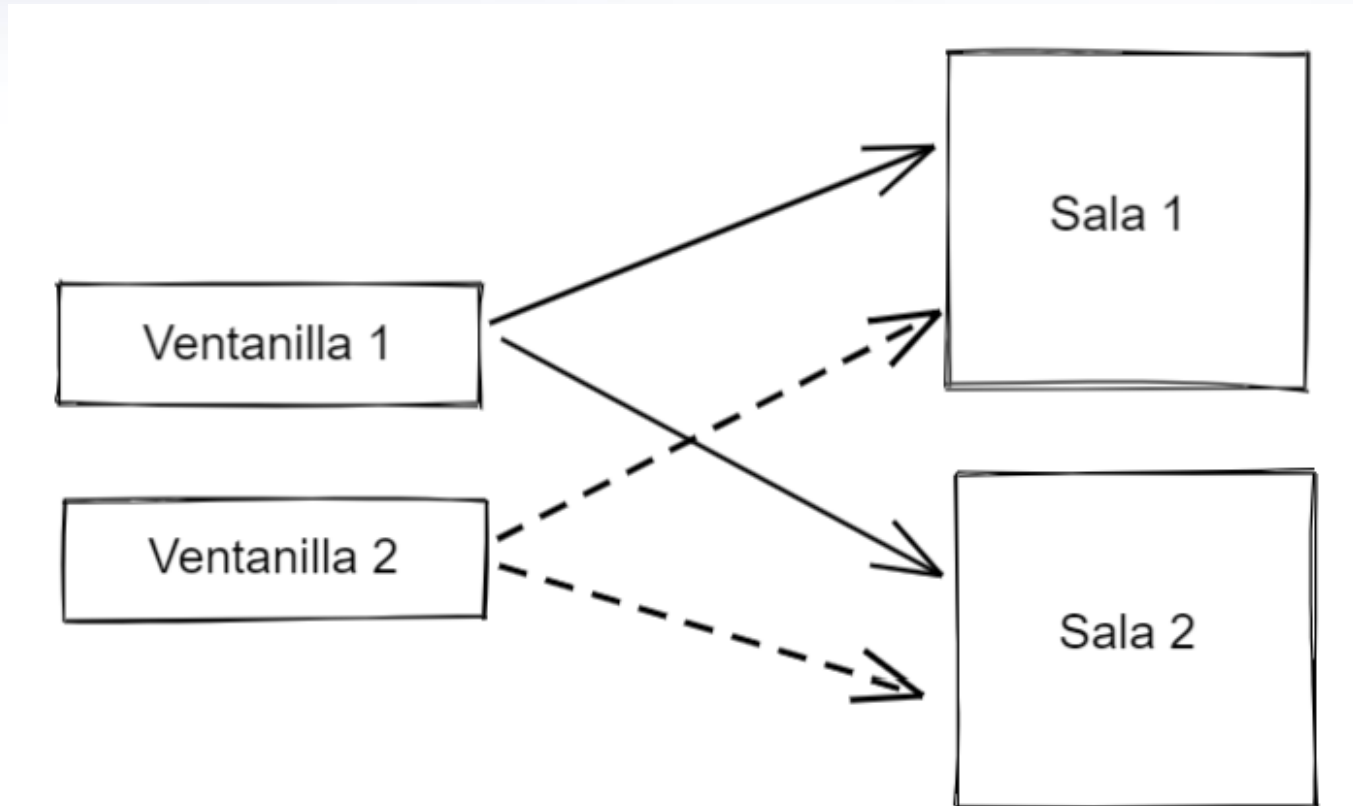
- Cuando se utiliza la palabra **synchronized** para proteger un bloque de código, debe pasar un objeto referencia como parámetro (llave).
- Normalmente se usa el argumento THIS, para referenciar el objeto que ejecuta ese método.
 - Puede usarse otro objeto como referencia.
- Por ej. si tiene dos atributos **independientes** en una clase compartida por multiples hilos debe sincronizar el acceso a ellas.

Arranging independent attributes in synchronized classes

- En el próximo ejercicio resolverá una situación de simulación de un cine con dos pantallas y dos oficinas de venta.
 - Cuando una oficina vende una entrada es para una pantalla no para ambas.
 - La cantidad de asientos libres de cada sala es INDEPENDIENTE.

Arranging independent attributes in synchronized classes

- Esquema del cine



Arranging independent attributes in synchronized classes

Paso a Paso:

- Crear la clase Cinema. Agregar 2 atributos que representarán los asientos vacíos de ambas salas.
 - Agregar a la clase Cinema 2 objetos adicionales que se usarán para el control del código SC.
 - Implementar el constructor.
 - Implementar el método sellTickets1. El mismo es llamado cuando se vende una entrada de esta sala. Y se utiliza el objeto controlCinema1 para SC.
 - Implementar el método sellTickets2. Idem con controlCinema2.

Arranging independent attributes in synchronized classes

- Implementar el método returnTickets1. Método que se llama cuando se devuelven entradas de la sala 1.
- Implementar el método returnTickets2. Idem.
- Implementar dos métodos que devolverán el número de vacantes de ambas salas.
- Implementar la clase TicketOffice1, runnable.
 - Declarar un objeto Cinema, y el constructor.
 - Implementar el método run(). Y simular algunas operaciones con ambas salas.
- Implementar idem TicketOffice2.

Arranging independent attributes in synchronized classes

- Implementar la clase Main.
 - Declarar y crear un objeto Cinema.
 - Crear un objeto de la clase TicketOffice1 y un Thread.
 - Idem TicketOffice2.
 - Lanzar ambos hilos.
 - Esperar la finalización de los hilos con join.
 - Escribir en la consola las vacantes de ambas salas.

Arranging independent attributes in synchronized classes

- Como funciona:
 - Cuando se utiliza la palabra synchronized para proteger un bloque de código se usa un objeto como parámetro.
 - JVM garantiza que solo un hilo puede acceder a todos los bloques de código protegidos con ese objeto.

Arranging independent attributes in synchronized classes

- Clase

TicketOffice1

```

Main.java Cinema.java TicketOffice1.java TicketOffice2.java
1 package com.packtpub.java7.concurrency.chapter2.recipe2.task;
2
3
4 * This class simulates a ticket office. It sell or return tickets.
5
6 public class TicketOffice1 implements Runnable {
7
8
9
10
11 * The cinema
12 private Cinema cinema;
13
14
15
16 * Constructor of the class
17 public TicketOffice1 (Cinema cinema) {
18     this.cinema=cinema;
19 }
20
21
22
23
24 * Core method of this ticket office. Simulates selling and returning tickets
25
26 @Override
27 public void run() {
28     cinema.sellTickets1(3);
29     cinema.sellTickets1(2);
30     cinema.sellTickets2(2);
31     cinema.returnTickets1(3);
32     cinema.sellTickets1(5);
33     cinema.sellTickets2(2);
34     cinema.sellTickets2(2);
35     cinema.sellTickets2(2);
36 }
37
```


Arranging independent attributes in synchronized classes

- Class

TicketOffice2

```

Main.java Cinema.java TicketOffice1.java TicketOffice2.java ✕
1 package com.packtpub.java7.concurrency.chapter2.recipe2.task;
2
4+ * This class simulates a ticket office. It sell or return tickets.
8 public class TicketOffice2 implements Runnable {
9
11+ * The cinema .
13 private Cinema cinema;
14
16+ * Constructor of the class.
19- public TicketOffice2(Cinema cinema){
20     this.cinema=cinema;
21 }
22
24+ * Core method of this ticket office. Simulates selling and returning tickets.
26- @Override
27 public void run() {
28     cinema.sellTickets2(2);
29     cinema.sellTickets2(4);
30     cinema.sellTickets1(2);
31     cinema.sellTickets1(1);
32     cinema.returnTickets2(2);
33     cinema.sellTickets1(3);
34     cinema.sellTickets2(2);
35     cinema.sellTickets1(2);
36 }
37
```

Arranging independent attributes in synchronized classes

Cinema

```

Main.java Cinema.java TicketOffice1.java TicketOffice2.java
3 public class Cinema {
4
5     * This two variables store the vacancies in two cinemas.
6     private long vacanciesCinema1;
7     private long vacanciesCinema2;
8
9
10
11     * Two objects for the synchronization. ControlCinema1 synchronizes the
12     private final Object controlCinema1, controlCinema2;
13
14
15     * Constructor of the class. Initializes the objects.
16     public Cinema(){
17         controlCinema1=new Object();
18         controlCinema2=new Object();
19         vacanciesCinema1=20;
20         vacanciesCinema2=20;
21     }
22
23     * This method implements the operation of sell tickets for the cinema 1.
24     public boolean sellTickets1 (int number) {
25         synchronized (controlCinema1) {
26             if (number<vacanciesCinema1) {
27                 vacanciesCinema1-=number;
28                 return true;
29             } else {
30                 return false;
31             }
32         }
33     }
34
35     * This method implements the operation of sell tickets for the cinema 2.
36     public boolean sellTickets2 (int number){
37         synchronized (controlCinema2) {
38             if (number<vacanciesCinema2) {
39                 vacanciesCinema2-=number;
40                 return true;
41             } else {
42                 return false;
43             }
44         }
45     }
46 }
```

Arranging independent attributes in synchronized classes

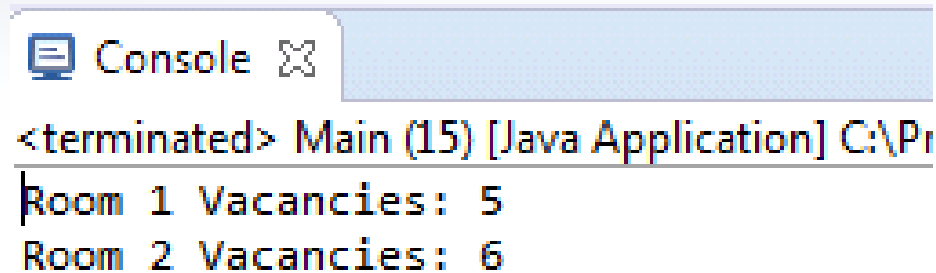
- Clase Main

```

Main.java  Cinema.java  TicketOffice1.java  TicketOffice2.java
1  package com.packtpub.java7.concurrency.chapter2.recipe2.core;
2
3  import com.packtpub.java7.concurrency.chapter2.recipe2.task.Cinema;
4
5
6
7
8  * Core class of the example. Creates a cinema and two threads for
9
10
11
12 public class Main {
13
14     * Main method of the example
15
16     public static void main(String[] args) {
17         // Creates a Cinema
18         Cinema cinema=new Cinema();
19
20         // Creates a TicketOffice1 and a Thread to run it
21         TicketOffice1 ticketOffice1=new TicketOffice1(cinema);
22         Thread thread1=new Thread(ticketOffice1,"TicketOffice1");
23
24         // Creates a TicketOffice2 and a Thread to run it
25         TicketOffice2 ticketOffice2=new TicketOffice2(cinema);
26         Thread thread2=new Thread(ticketOffice2,"TicketOffice2");
27
28         // Starts the threads
29         thread1.start();
30         thread2.start();
31
32         try {
33             // Waits for the finalization of the threads
34             thread1.join();
35             thread2.join();
36         } catch (InterruptedException e) {
37             e.printStackTrace();
38         }
39
40         // Print the vacancies in the cinemas
41         System.out.printf("Room 1 Vacancies: %d\n",cinema.getVacanciesCinema1());
42         System.out.printf("Room 2 Vacancies: %d\n",cinema.getVacanciesCinema2());
43     }
44 }
45
46
47 }
```

Arranging independent attributes in synchronized classes

- Resultados Esperados



The screenshot shows a console window titled "Console" with a close button. The output text is as follows:

```
<terminated> Main (15) [Java Application] C:\Pr  
Room 1 Vacancies: 5  
Room 2 Vacancies: 6
```

Synchronized con un objeto: Detalles

- Se vió un ejercicio donde el acceso a la venta de tickets de dos salas se sincronizaba a través de la primitiva `synchronized()`; enviando un object como llave.
- Veremos a continuación los detalles respecto a posibles errores al enviar diferentes objetos como llaves.
- <https://www.securecoding.cert.org/confluence/display/java/LCK01-J.+Do+not+synchronize+on+objects+that+may+be+reused>

Synchronized con un objeto: Detalles

- La confusión puede surgir del concepto que casi todos los objetos en java derivan de la clase “object”, por lo cual podrían ser utilizados como “key” para sincronizar un bloque de código.
- Es importante mencionar que en java como en otros lenguajes orientados a objetos un objetivo es optimizar el uso de memoria, la creación por ej. de un **Integer**, no es más que un wrapper, apuntador o manipulador del objeto real al que apunta.

Synchronized con un objeto: Detalles

- Cuando se sincroniza con un WRAPPER de un tipo primitivo por ejemplo un INTEGER, en realidad se utiliza el KEY (semáforo binario) del objeto valor que está apuntando.
- Cuando en el interior del bloque de código se cambia el valor de ese WRAPPER, se utiliza el key del nuevo valor apuntado.
- Puntualmente en el caso del ejemplo del cine, si se hace la sincronización con los dos valores INTEGER de vacantes, al modificar internamente el valor de vacantes se puede dar (en 1000 ejecuciones, 10 veces aprox. por ej) que queden ambas llaves apuntando al mismo valor, por lo que pueden acceder ambos hilos al mismo bloque de código y tenemos inconsistencia en los resultados en algunas corridas.

Synchronized con un objeto: Detalles

- Veremos los siguientes casos de sincronización:
 - Caso1
Del libro donde la sincronización se realiza sobre 2 **objects** creados para tal fin.
 - Caso 2
Consultado por alumno en clases, donde se sincroniza con la misma variable VACANCIES (Wrapper **Integer**) enviada al metodo sincronizado para operar.
 - Caso 3
Donde se sincroniza con un **arreglo** de elementos, utilizando el arreglo como objeto llave.
 - Caso 4
Donde se sincroniza con un **elemento del arreglo** del caso anterior.
 - Caso 5
Donde se sincroniza con un **object nuevo**, donde internamente tiene un atributo y metodos para las operaciones.

Synchronized con un objeto: Detalles

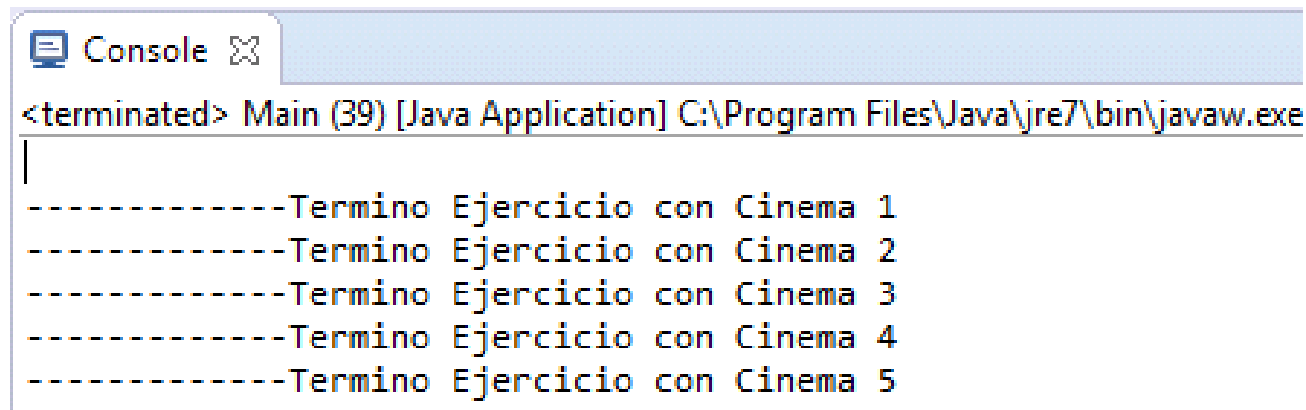
- Veremos los siguientes casos de sincronización:
 - Caso 1
Del libro donde la sincronización se realiza sobre 2 **objects** creados para tal fin. Resultado OK.
 - Caso 2
Consultado por alumno en clases, donde se sincroniza con la misma variable VACANCIES (Wrapper **Integer**) enviada al metodo sincronizado para operar. Resultado ERRONEO.
 - Caso 3
Donde se sincroniza con un **arreglo** de elementos, utilizando el arreglo como objeto llave. Resultado OK.
 - Caso 4
Donde se sincroniza con un **elemento del arreglo** del caso anterior. Resultado ERRONEO.
 - Caso 5
Donde se sincroniza con un **object nuevo**, donde internamente tiene un atributo y metodos para las operaciones. Resultado OK.

Synchronized con un objeto: Detalles

- Se pueden observar los detalles de la implementación en el proyecto SynchronizedObject.
 - El mismo tiene la clase Main, donde se crea un arreglo de interfaces y se le asigna un objeto de cada tipo de los 5 casos enumerados anteriormente.
 - Luego con 2 bucles se ejecutan las veces seleccionadas las corridas para cada caso de los 5 descriptos.
 - En el bucle interno se analiza que los resultados sean correctos, ante una inconsistencia se imprime NO ANDA, en la consola.

Synchronized con un objeto: Detalles

- A continuación se observa el resultado de ejecutar 10 veces cada grupo de instrucciones de los tickets office...como puede observarse...todo parece funcionar OK!



```
Console X
<terminated> Main (39) [Java Application] C:\Program Files\Java\jre7\bin\javaw.exe
|
-----Termino Ejercicio con Cinema 1
-----Termino Ejercicio con Cinema 2
-----Termino Ejercicio con Cinema 3
-----Termino Ejercicio con Cinema 4
-----Termino Ejercicio con Cinema 5
```

Synchronized con un objeto: Detalles

- A continuación se observa el resultado de ejecutar 100000 veces cada grupo de instrucciones de los tickets office...
Cuidado...el testing puede engañarnos...

```
Console
<terminated> Main (39) [Java Application] C:\Program Files\Java\jre7\bin\javaw.exe

-----Termino Ejercicio con Cinema 1
NO ANDA
NO ANDA
NO ANDA
NO ANDA
NO ANDA
NO ANDA
NO ANDA
NO ANDA
NO ANDA

-----Termino Ejercicio con Cinema 2
-----Termino Ejercicio con Cinema 3
NO ANDA

-----Termino Ejercicio con Cinema 4
-----Termino Ejercicio con Cinema 5
```

Condición de sincronización en Java

- La utilización de **synchronized** proporciona un mecanismo de seguridad e integridad para el acceso a datos compartidos.
- Hay que tener en cuenta también la **sincronización y comunicación** entre los distintos threads.
 - Para implementar las esperas por ocurrencias de eventos y notificaciones de los mismos, Java proporciona tres métodos:

–**wait()**

notify()

notifyAll()

Condición de sincronización en Java

- *wait()*: el hilo en curso abandona la exclusión mutua sobre el objeto (el cerrojo) y se va al estado «*en espera*»
- Permanecerá ahí hasta que otro hilo lo despierte mediante los métodos *notify()* o *notifyAll()*.
- El lugar donde se queda esperando el thread recibe el nombre de *conjunto de espera* o *wait set*.

Condición de sincronización en Java

- *notify()*: un thread arbitrario del conjunto de espera es seleccionado por el planificador para pasar al estado listo.
- *notifyAll()*: todos los threads del conjunto de espera son puestos en el estado listo.
- Estos métodos están definidos en la clase Object y por tanto son heredados por todas las clases de forma implícita.

Synchronizing a block of code with Lock

- Java provee otro mecanismo para la sincronización del acceso a bloques de código.
- Este mecanismo presenta las siguientes ventajas:
 - Permite la estructuración de bloques de manera más flexible.
 - Este mecanismo es más potente, presenta funcionalidades mejoradas como el `tryLock()`.
Pregunto si está tomado el lock (no me bloqueo).
 - La interface `lock` permite separación de operaciones `read` y `write`.
 - Mejor performance.

Synchronizing a block of code with Lock

- En el próximo ejercicio, se aprenderá a utilizar la herramienta LOCK, para sincronizar un bloque de código y se creará una sección crítica usando locks (reentrants locks).
- Clases a utilizar:
 - Printqueue. Implementará una cola de impresión.
 - Job.
 - Main.

Synchronizing a block of code with Lock

- Clase Main: lanza 10 hilos ejecutando objetos de tipo Job en paralelo. Estos envían trabajos a imprimir al mismo tiempo.



```
1 package com.packtpub.java9.concurrency.cookbook.chapter02.recipe03.core;
2 import com.packtpub.java9.concurrency.cookbook.chapter02.recipe03.task.Job;
3 import com.packtpub.java9.concurrency.cookbook.chapter02.recipe03.task.PrintQueue;
4
5 public class Main {
6
7     public static void main (String args[]) {
8         PrintQueue printQueue = new PrintQueue();
9
10        // Creates ten Threads
11        Thread thread[] = new Thread[10];
12        for (int i = 0; i < 10; i++) {
13            thread[i] = new Thread(new Job(printQueue), "Thread " + i);
14        }
15
16        // Starts the Threads
17        for (int i = 0; i < 10; i++) {
18            thread[i].start();
19        }
20    }
21 }
```

Synchronizing a block of code with Lock

- Clase Job: simula una tarea que envía un documento a la impresora.

```

Main.java × PrintQueue.java × Job.java ×
1  package com.packtpub.java9.concurrency.cookbook.chapter02.recipe03.task;
2
3  public class Job implements Runnable {
4
5      private PrintQueue printQueue;
6
7      public Job(PrintQueue printQueue) { this.printQueue=printQueue; }
10
11      @Override
12      public void run() {
13          System.out.printf("%s: Going to print a document\n",Thread.currentThread().getName());
14          printQueue.printJob(new Object());
15          System.out.printf("%s: The document has been printed\n",Thread.currentThread().getName());
16      }
17  }
```

Synchronizing a block of code with Lock

- PrintQueue: Esta clase simula una cola de impresión. Crea un objeto de tipo lock para controlar el acceso.

```
1 package com.packtpub.java9.concurrency.cookbook.chapter02.recipe03.task;
2
3 import java.util.concurrent.locks.Lock;
4 import java.util.concurrent.locks.ReentrantLock;
5
6 public class PrintQueue {
7
8     private Lock queueLock;
9
10    public PrintQueue() { queueLock = new ReentrantLock(); }
11
12
13
14    public PrintQueue(boolean fairMode) { queueLock = new ReentrantLock(fairMode); }
15
16
17
18    public void printJob(Object document) {
19        queueLock.lock();
20        try {
21            Long duration = (long) (Math.random() * 10000);
22            System.out.printf("%s: PrintQueue: Printing a Job during %d seconds\n", Thread.currentThread().getName(),
23                               (duration / 1000));
24            Thread.sleep(duration);
25        } catch (InterruptedException e) {
26            e.printStackTrace();
27        } finally {
28            queueLock.unlock();
29        }
30    }
31 }
```

Synchronizing a block of code with Lock

- La salida del programa.



```
1: Main x
/Library/Java/JavaVirtualMachines/jdk1.8.0_191.jdk/Contents/Home/bin/java ...
Thread 0: Going to print a document
Thread 9: Going to print a document
Thread 8: Going to print a document
Thread 7: Going to print a document
Thread 6: Going to print a document
Thread 5: Going to print a document
Thread 4: Going to print a document
Thread 3: Going to print a document
Thread 2: Going to print a document
Thread 1: Going to print a document
Thread 0: PrintQueue: Printing a Job during 0 seconds
Thread 0: The document has been printed
Thread 9: PrintQueue: Printing a Job during 1 seconds
Thread 9: The document has been printed
Thread 8: PrintQueue: Printing a Job during 7 seconds
Thread 8: The document has been printed
Thread 7: PrintQueue: Printing a Job during 8 seconds
```

Locks - Details

- Bloque de código FINALLY. Obliga la ejecución del mismo, aun cuando hay una excepción que haga salir del bloque try catch.
- Con la flexibilidad de los LOCKS, viene la RESPONSABILIDAD del programador...Cuidado con el orden de los LOCKS, dos hilos tomando locks en orden diferente pueden terminar en deadlock.
- Los locks reentrants, permite el llamado a métodos recursivos y tienen características avanzadas Ver javadoc.

Synchronized o LOCKS

- Keep it simple **S** KISS.
- Dejar **reentrantsLocks** para casos en los cuales necesitamos las capacidades especiales que no conseguimos con un **bloque synchronized**:
 - Consultar sin bloquearse por el lock trylock.
 - Adquirir un bloqueo interrumpible.
 - Intentar adquirir un lock por una determinada cantidad de tiempo Trylock temporizados.
 - Implementacion de fairness en colas de espera.
 - Sincronizar bloques no estructurados.

En Resumen...

- Sección Crítica
- Synchronized en métodos
- Synchronized en bloques de código
- Detalles de la sincronización con objetos como llave
- Métodos de comunicación entre hilos
- Locks